

CS-499 Milestone Three: Enhancement Narrative

Enhancement Two: Algorithms and Data Structures

Michael King

7/24/2026

Artifact Description

The artifact is the Grazioso Salvare Animal Shelter Dashboard created for CS 340: Advanced Programming Concepts. It is an interactive web application built with Python, MongoDB, and Plotly Dash. The application functionality consists of a CRUD module, `animalshelter.py`, that interacts with a collection of 10,000 animal shelter outcomes in a Mongo database, and a Dash-based dashboard (`app.py/ProjectTwoDashboard.ipynb`) that allows a user to filter animals based on rescue type (Water Rescue, Mountain/Wilderness, Disaster/Tracking), see the filtered data in an interactive data table, see a breed distribution pie chart, and see a geolocation map of a selected animal. The original version of this artifact was created earlier in the Computer Science program and was adopted into the ePortfolio during the CS 499 Module One planning process.

Why This Artifact Was Selected

This artifact was selected for the Algorithms and Data Structures category mainly because it provided distinct opportunities to optimize the algorithms presented. The original artifact implemented each rescue type selection by issuing a full, unfiltered collection scan over the entire MongoDB collection, retrieving the data to the client, then applying Boolean masks over the pandas DataFrame of the query data to filter out unnecessary records. This meant that

every filter request was an $O(n)$ collection scan and then a re-filtering of the same data held in memory, even when it was a filter that had been requested a matter of seconds before. As the original dashboard already addressed the user-facing aspect of displaying and filtering data on animal shelters, the enhancement improved the efficiency, scalability, and responsiveness of this existing application.

The enhancement targeted three specific, complementary improvements to this process. The first was a compound index on `breed`, `sex_upon_outcome`, and `age_upon_outcome_in_weeks`, the three fields every rescue-type filter searches on, along with a separate index on `location_lat` and `location_long` to enable the geolocation map to be rendered, so Mongo will be able to perform the index-assisted lookup, $O(\log n)$, instead of a complete collection scan, $O(n)$, for this task. The second improvement was a new method, `read_with_filter(rescue_type)`, which replaces client-side pandas filtering with a MongoDB aggregation pipeline (`$match`, `$project`, `$sort`), where filtering, field selection, and sorting all happen on the server side outside of Python, not after the collection has been fully streamed down to a Python DataFrame. The third improvement was a dictionary cache, `query_cache`, which saves the result of each rescue-type query after the first run, so that a subsequent request for the same data can be an $O(1)$ dictionary lookup, rather than an $O(n)$ query, with its cache automatically cleared on the next `create()`, `update()`, or `delete()` call to ensure all data remains consistent at all times. This set of improvements exemplifies the process of analyzing a production system, pinpointing the true bottleneck, and applying multiple algorithmic solutions, from indexing through server-side query processing to caching, each of which addresses a small, distinct facet of the problem of speed.

Course Outcome Alignment

The completion of the artifact satisfied the outcome of designing and implementing computing solutions by employing algorithmic techniques to make design tradeoffs, because each of the three enhancement decisions was prefaced in the course by an explicit before/after complexity statement, from $O(n)$ to $O(\log n)$ to $O(1)$, for repeated queries. The artifact also satisfied the course outcome of employing well-grounded and creative techniques and tools to realize solutions that provide value, because indexing, aggregation pipelines, and memoization are well-founded, standard, and pragmatic techniques used in concurrency. No modifications were made to the original outcome coverage plan; the improvement was implemented as designed in Module One.

Challenges Faced

The implementation of the enhancement illustrated how much optimization in a full-stack application actually occurred at the intersection of the application and the database, rather than entirely within a single function. Refactoring filtering into an aggregation pipeline involved conceptualizing which bits of the pandas filtering code (breed lists, sex, age ranges) would translate into a set of $\$in$, $\$gte/\lte queries, and verifying that the compound index aligned with any fields those queries would depend on.

One particular problem revealed only through hands-on testing and not inspection of the code was that after the dashboard's callback was wired to call `read_with_filter()`, the `rec_num` column had disappeared from the data table in every case, including the "Reset All Animals" view, even though it was still being displayed correctly in the original version running on the same server. Regression traced back to the aggregation pipeline `$project` stage provided the answer: `$project` in inclusion mode is non-null fields only, and result fields have to be explicitly added with 1 to `PROJECTION_FIELDS`, but `rec_num` was missing. Because the original

filtering in pandas sliced columns from an existing DataFrame that already had every column, there was no failure mode for this sort of null-value data loss with the original code; instead, this was a subtle, field-by-field database loss risk linked exclusively to context-specific projection implementations. This was fixed by just inserting 'rec-num': 1 into the projection, re-executing all three rescue-type filters plus the reset view, and confirming that it worked again against the live MongoDB instance.

Conclusion

This experience was an excellent reminder that enhancement work is not done when the code finally runs without errors; it is done when the output has been checked column by column against the source value. It also reinforced the need to run the original and enhanced versions side by side while developing, as the difference pointed out here was only visible when directly comparing the two dashboards visually.